

# 10 Symbolic Execution

July 1, 2026

## 1 (Dynamic) Symbolic Execution

This chapter is an excerpt of Andreas Zeller's excellent [Fuzzing Book](#), cutting some corners.

```
[1]: def factorial(n):  
    if n < 0:  
        return None  
  
    if n == 0:  
        return 1  
  
    if n == 1:  
        return 1  
  
    v = 1  
    while n != 0:  
        v = v * n  
        n = n - 1  
  
    return v
```

```
[2]: factorial(5)
```

```
[2]: 120
```

### 1.1 SMT Solver

Dynamic Symbolic Execution uses modern constraint solving techniques to derive input values, in particular it is usually built using Satisfiability Modulo Theories (SMT) solvers. These are built on top of regular SAT solvers, which determine whether first order logic formulas are satisfiable. SMT solvers extend these with additional background theories, such as the theory of integers.

One of the best SMT solvers available is (<https://github.com/Z3Prover/z3>) [<https://github.com/Z3Prover/z3>].

```
[3]: import z3
```

```
[4]: z3.set_option('timeout', 30 * 1000) # milliseconds
```

Constraints are formulas using symbolic variables. For example, to create a symbolic variable representing integer `n`, we can use the Z3-API as follows.

```
[5]: zn = z3.Int('n')
```

Symbolic variables can be used to construct constraints.

```
[6]: zn < 0
```

```
[6]: n < 0
```

```
[7]: z3.Not(zn < 0)
```

```
[7]: Not(n < 0)
```

A constraint is solved by finding an assignment of values to the symbolic variables such that the formula is satisfied.

```
[8]: z3.solve(z3.Not(zn < 0))
```

```
[n = 0]
```

```
[9]: z3.solve(zn < 0)
```

```
[n = -1]
```

## 1.2 A Concolic Tracer

DSE, or concolic execution (concolic = concrete+symbolic), consists of collecting symbolic path conditions during concrete executions. This means that during execution we need to track (1) the symbolic context, consisting of the symbolic variables and their symbolic values, and (2) the path conditions that describe the control flow decisions made along an execution.

Let us now define a class to collect symbolic variables and path conditions during an execution. The idea is to have a ConcolicTracer class that is invoked in a with block. To execute a function while tracing its path conditions, we need to transform its arguments, which we do by invoking functions through a [] item access.

This is a typical usage of a ConcolicTracer:

```
with ConcolicTracer as _:  
    _.[function](args, ...)
```

After execution, we can access the symbolic variables in the decls attribute:

```
_.decls
```

whereas the path attribute lists the precondition paths encountered:

```
_.path
```

The context attribute contains a pair of declarations and paths:

```
_.context
```

```
[10]: import inspect
```

Let us now implement ConcolicTracer constructor of a accepts a single context argument which contains the declarations for the symbolic variables seen so far, and path conditions seen so far. We only need this in case of nested contexts.

```
[11]: class ConcolicTracer:
        """Trace function execution, tracking variables and path conditions"""

        def __init__(self, context=None):
            """Constructor."""
            self.context = context if context is not None else ({}, [])
            self.decls, self.path = self.context
```

We add the enter and exit methods for the with block.

```
[12]: class ConcolicTracer(ConcolicTracer):
        def __enter__(self):
            return self

        def __exit__(self, exc_type, exc_value, tb):
            return
```

We use introspection to determine the arguments to the function, which is hooked into the getitem() method.

```
[13]: class ConcolicTracer(ConcolicTracer):
        def __getitem__(self, fn):
            self.fn = fn
            self.fn_args = {i: None for i in inspect.signature(fn).parameters}
            return self
```

Finally, the function itself is invoked using the call method.

```
[14]: class ConcolicTracer(ConcolicTracer):
        def __call__(self, *args):
            self.result = self.fn(*self.concolic(args))
            return self.result
```

For now, we define concolic() as a transparent function. It will be modified to produce symbolic variables later.

```
[15]: class ConcolicTracer(ConcolicTracer):
        def concolic(self, args):
            return args
```

We now have things in place for tracing functions:

```
[16]: with ConcolicTracer() as _:
        _[factorial](1)
```

And for retrieving results (but not actually computing them):

```
[17]: _ .decls
```

```
[17]: {}
```

```
[18]: _ .path
```

```
[18]: []
```

Both decls and path attributes will be set by concolic proxy objects, which we define next.

### 1.3 Concolic Proxy Objects

We now define the concolic proxy objects that can be used for concolic tracing. First, we define the `zproxy_create()` method that given a class name, correctly creates an instance of that class, and the symbolic corresponding variable, and registers the symbolic variable in the context information context.

```
[19]: def zproxy_create(cls, z_type, z3var, context, z_name, v=None):  
    z_value = cls(context, z3var(z_name), v)  
    context[0][z_name] = z_type # add to decls  
    return z_value
```

#### 1.3.1 A Proxy Class for Booleans

First, we define the `zbool` class which is used to track the predicates encountered. It is a wrapper class that contains both symbolic (`z`) as well as concrete (`v`) values. The concrete value is used to determine which path to take, and the symbolic value is used to collect the predicates encountered.

The initialization is done in two parts. The first one is using `zproxy_create()` to correctly initialize and register the shadow symbolic variable corresponding to the passed argument. This is used exclusively when the symbolic variable needs to be initialized first. In all other cases, the constructor is called with the preexisting symbolic value.

```
[20]: class zbool:  
    @classmethod  
    def create(cls, context, z_name, v):  
        return zproxy_create(cls, 'Bool', z3.Bool, context, z_name, v)  
  
    def __init__(self, context, z, v=None):  
        self.context = context  
        self.z = z  
        self.v = v  
        self.decl, self.path = self.context
```

Here is how it is used. We create a symbolic variable `my_bool_arg` with a value of `True` in the current context of the concolic tracer:

```
[21]: with ConcolicTracer() as _:  
    val = zbool.create(_ .context, 'my_bool_arg', True)
```

We can now access the symbolic name in the z attribute:

```
[22]: val.z
```

```
[22]: my_bool_arg
```

The value is in the v attribute:

```
[23]: val.v
```

```
[23]: True
```

Note that the context of the enclosing `ConcolicTracer()` is automatically updated (via `zproxy_create()`) to hold the variable declarations and types:

```
[24]: _.context
```

```
[24]: ({'my_bool_arg': 'Bool'}, [])
```

The context can also be reached through the context attribute; both point to the same data structure.

```
[25]: val.context
```

```
[25]: ({'my_bool_arg': 'Bool'}, [])
```

### 1.3.2 Negation of Encoded formula

The `zbool` class allows negation of its concrete and symbolic values.

```
[26]: class zbool(zbool):  
      def __not__(self):  
          return zbool(self.context, z3.Not(self.z), not self.v)
```

Here is how it can be used.

```
[27]: with ConcolicTracer() as _:  
      val = zbool.create(_.context, 'my_bool_arg', True).__not__()
```

```
[28]: val.z
```

```
[28]: Not(my_bool_arg)
```

```
[29]: val.v
```

```
[29]: False
```

```
[30]: _.context
```

```
[30]: ({'my_bool_arg': 'Bool'}, [])
```

### 1.3.3 Registering Predicates on Conditionals

The `zbool` class is being used to track Boolean conditions that arise during program execution. It tracks such conditions by registering the corresponding symbolic expressions in the context as soon as it is evaluated. On evaluation, the `__bool__()` method is called; so we can hook into this one:

```
[31]: class zbool(zbool):
      def __bool__(self):
          r, pred = (True, self.z) if self.v else (False, z3.Not(self.z))
          self.path.append(pred)
          return r
```

The `zbool` class can be used to keep track of Boolean values and conditions encountered during the execution. For example, we can encode the conditions encountered by Line 6 in `factorial()` as follows:

First, we define the concrete value (`ca`), and its shadow symbolic variable (`za`).

```
[32]: ca = 5
      za = z3.Int('a')
```

Then, we wrap it in `zbool`, and use it in a conditional, forcing the conditional to be registered in the context.

```
[33]: with ConcolicTracer() as _:
      if zbool(_.context, za == z3.IntVal(5), ca == 5):
          print('success')
```

success

We can retrieve the registered conditional as follows.

```
[34]: _.path
```

```
[34]: [5 == a]
```

### 1.3.4 A Proxy Class for Integers

Next, we define a symbolic wrapper `zint` for `int`. This class keeps track of the `int` variables used and the predicates encountered in context. Finally, it also keeps the concrete value so that it can be used to determine the path to take. As the `zint` extends the primitive `int` class, we have to define a new method to open it for extension.

```
[35]: class zint(int):
      def __new__(cls, context, zn, v, *args, **kw):
          return int.__new__(cls, v, *args, **kw)
```

As in the case of `zbool`, the initialization takes place in two parts. The first using `create()` if a new symbolic argument is being registered, and then the usual initialization.

```
[36]: class zint(zint):
    @classmethod
    def create(cls, context, zn, v=None):
        return zproxy_create(cls, 'Int', z3.Int, context, zn, v)

    def __init__(self, context, z, v=None):
        self.z, self.v = z, v
        self.context = context
```

The int value of a zint object is its concrete value.

```
[37]: class zint(zint):
    def __int__(self):
        return self.v

    def __pos__(self):
        return self.v
```

Using these proxies is as follows.

```
[38]: with ConcolicTracer() as _:
    val = zint.create(_.context, 'int_arg', 0)
```

```
[39]: val.z
```

```
[39]: int_arg
```

```
[40]: val.v
```

```
[40]: 0
```

```
[41]: _.context
```

```
[41]: ({'int_arg': 'Int'}, [])
```

The zint class is often used to do arithmetic with, or compare to other ints. These ints can be either a variable or a constant value. We define a helper method `_zv()` that checks what kind of int a given value is, and produces the correct symbolic equivalent.

```
[42]: class zint(zint):
    def _zv(self, o):
        return (o.z, o.v) if isinstance(o, zint) else (z3.IntVal(o), o)
```

It can be used as follows

```
[43]: with ConcolicTracer() as _:
    val = zint.create(_.context, 'int_arg', 0)
```

```
[44]: val._zv(0)
```

```
[44]: (0, 0)
```

```
[45]: val._zv(val)
```

```
[45]: (int_arg, 0)
```

### 1.3.5 Equality between Integers

Two integers can be compared for equality using `ne` and `eq`.

```
[46]: class zint(zint):
    def __ne__(self, other):
        z, v = self._zv(other)
        return zbool(self.context, self.z != z, self.v != v)

    def __eq__(self, other):
        z, v = self._zv(other)
        return zbool(self.context, self.z == z, self.v == v)
```

We also define `req` using `eq` in case the int being compared is on the left hand side.

```
[47]: class zint(zint):
    def __req__(self, other):
        return self.__eq__(other)
```

It can be used as follows.

```
[48]: with ConcolicTracer() as _:
    ia = zint.create(_.context, 'int_a', 0)
    ib = zint.create(_.context, 'int_b', 0)
    v1 = ia == ib
    v2 = ia != ib
    v3 = 0 != ib
    print(v1.z, v2.z, v3.z)
```

```
int_a == int_b int_a != int_b 0 != int_b
```

### 1.3.6 Comparisons between Integers

Integers can also be compared for ordering, and the methods for this are defined below.

```
[49]: class zint(zint):
    def __lt__(self, other):
        z, v = self._zv(other)
        return zbool(self.context, self.z < z, self.v < v)

    def __gt__(self, other):
        z, v = self._zv(other)
        return zbool(self.context, self.z > z, self.v > v)
```

We use the comparisons and equality operators to provide the other missing operators.

```
[50]: class zint(zint):
    def __le__(self, other):
        z, v = self._zv(other)
        return zbool(self.context, z3.Or(self.z < z, self.z == z),
                        self.v < v or self.v == v)

    def __ge__(self, other):
        z, v = self._zv(other)
        return zbool(self.context, z3.Or(self.z > z, self.z == z),
                        self.v > v or self.v == v)
```

These functions can be used as follows.

```
[51]: with ConcolicTracer() as _:
    ia = zint.create(_.context, 'int_a', 0)
    ib = zint.create(_.context, 'int_b', 1)
    v1 = ia > ib
    v2 = ia < ib
    print(v1.z, v2.z)
    v3 = ia >= ib
    v4 = ia <= ib
    print(v3.z, v4.z)
```

int\_a > int\_b int\_a < int\_b

Or(int\_a > int\_b, int\_a == int\_b) Or(int\_a < int\_b, int\_a == int\_b)

### 1.3.7 Binary Operators for Integers

We implement relevant arithmetic operators for integers as described in the Python documentation. (The commented out operators are not directly available for z3.ArithRef. They need to be implemented separately if needed.)

```
[52]: INT_BINARY_OPS = [
    '__add__',
    '__sub__',
    '__mul__',
    '__truediv__',
    # '__div__',
    '__mod__',
    # '__divmod__',
    '__pow__',
    # '__lshift__',
    # '__rshift__',
    # '__and__',
    # '__xor__',
    # '__or__',
    '__radd__',
```

```

    '__rsub__',
    '__rmul__',
    '__rtruediv__',
    # '__rdiv__',
    '__rmod__',
    # '__rdivmod__',
    '__rpow__',
    # '__rlshift__',
    # '__rrshift__',
    # '__rand__',
    # '__rxor__',
    # '__ror__',
]

```

```

[53]: def make_int_binary_wrapper(fname, fun, zfun):
    def proxy(self, other):
        z, v = self._zv(other)
        z_ = zfun(self.z, z)
        v_ = fun(self.v, v)
        if isinstance(v_, float):
            # we do not implement float results yet.
            assert round(v_) == v_
            v_ = round(v_)
        return zint(self.context, z_, v_)

    return proxy

```

```

[54]: INITIALIZER_LIST = []

```

```

[55]: def initialize():
    for fn in INITIALIZER_LIST:
        fn()

```

```

[56]: def init_concolic_1():
    for fname in INT_BINARY_OPS:
        fun = getattr(int, fname)
        zfun = getattr(z3.ArithRef, fname)
        setattr(zint, fname, make_int_binary_wrapper(fname, fun, zfun))

```

```

[57]: INITIALIZER_LIST.append(init_concolic_1)

```

```

[58]: init_concolic_1()

```

```

[59]: with ConcolicTracer() as _:
    ia = zint.create(_.context, 'int_a', 0)
    ib = zint.create(_.context, 'int_b', 1)
    print((ia + ib).z)

```

```

print((ia + 10).z)
print((11 + ib).z)
print((ia - ib).z)
print((ia * ib).z)
print((ia / ib).z)
print((ia ** ib).z)

```

```

int_a + int_b
int_a + 10
11 + int_b
int_a - int_b
int_a*int_b
int_a/int_b
int_a**int_b

```

### 1.3.8 Integer Unary Operators

We also implement the relevant unary operators as below.

```

[60]: INT_UNARY_OPS = [
    '__neg__',
    '__pos__',
    # '__abs__',
    # '__invert__',
    # '__round__',
    # '__ceil__',
    # '__floor__',
    # '__trunc__',
]

```

```

[61]: def make_int_unary_wrapper(fname, fun, zfun):
    def proxy(self):
        return zint(self.context, zfun(self.z), fun(self.v))

    return proxy

```

```

[62]: def init_concolic_2():
    for fname in INT_UNARY_OPS:
        fun = getattr(int, fname)
        zfun = getattr(z3.ArithRef, fname)
        setattr(zint, fname, make_int_unary_wrapper(fname, fun, zfun))

```

```

[63]: INITIALIZER_LIST.append(init_concolic_2)

```

```

[64]: init_concolic_2()

```

We can use the unary operators we defined above as follows:

```
[65]: with ConcolicTracer() as _:
        ia = zint.create(_context, 'int_a', 0)
        print((-ia).z)
        print((+ia).z)
```

```
-int_a
int_a
```

### 1.3.9 Using an Integer in a Boolean Context

An integer may be converted to a boolean context in conditionals or as part of boolean predicates such as or, and and not. In these cases, the **bool()** method gets called. Unfortunately, this method requires a primitive boolean value. Hence, we force the current integer formula to a boolean predicate and register it in the current context.

```
[66]: class zint(zint):
        def __bool__(self):
            # return zbool(self.context, self.z, self.v) <-- not allowed
            # force registering boolean condition
            if self != 0:
                return True
            return False
```

It is used as follows

```
[67]: with ConcolicTracer() as _:
        za = zint.create(_context, 'int_a', 1)
        zb = zint.create(_context, 'int_b', 0)
        if za and zb:
            print(1)
```

```
[68]: _context
```

```
[68]: ({'int_a': 'Int', 'int_b': 'Int'}, [0 != int_a, Not(0 != int_b)])
```

### 1.3.10 Generating fresh names

While using the proxy classes, we often will have to generate new symbolic variables, with names that have not been used before. For this, we define `fresh_name()` that always generates unique integers for names.

```
[69]: COUNTER = 0
```

```
[70]: def fresh_name():
        global COUNTER
        COUNTER += 1
        return COUNTER
```

It can be used as follows:

```
[71]: fresh_name()
```

```
[71]: 1
```

```
[72]: def reset_counter():  
      global COUNTER  
      COUNTER = 0
```

```
[73]: class ConcolicTracer(ConcolicTracer):  
      def __enter__(self):  
          reset_counter()  
          return self  
  
      def __exit__(self, exc_type, exc_value, tb):  
          return
```

### 1.3.11 Translating Arguments to Concolic Proxies

We had previously defined `concolic()` as a transparent function. We now provide the full implementation of this function. It inspects a given function's parameters, and infers the parameter types from the concrete arguments passed in. It then uses this information to instantiate the correct proxy classes for each argument.

```
[74]: class ConcolicTracer(ConcolicTracer):  
      def concolic(self, args):  
          my_args = []  
          for name, arg in zip(self.fn_args, args):  
              t = type(arg).__name__  
              zwrap = globals()['z' + t]  
              vname = "%s_%s_%s_%s" % (self.fn.__name__, name, t, fresh_name())  
              my_args.append(zwrap.create(self.context, vname, arg))  
              self.fn_args[name] = vname  
          return my_args
```

This is how it gets used:

```
[75]: with ConcolicTracer() as _:  
      print(_[factorial](5))
```

```
120
```

With the new `concolic()` method, the arguments to the `factorial` are correctly associated with symbolic variables, which allows us to retrieve the predicates encountered.

```
[76]: _.context
```

```
[76]: ({'factorial_n_int_1': 'Int'},  
      [Not(0 > factorial_n_int_1),  
      Not(0 == factorial_n_int_1),
```

```

Not(1 == factorial_n_int_1),
0 != factorial_n_int_1,
0 != factorial_n_int_1 - 1,
0 != factorial_n_int_1 - 1 - 1,
0 != factorial_n_int_1 - 1 - 1 - 1,
0 != factorial_n_int_1 - 1 - 1 - 1 - 1,
Not(0 != factorial_n_int_1 - 1 - 1 - 1 - 1 - 1 - 1))

```

### 1.3.12 Evaluating the Concolic Expressions

We define `zeval()` to solve the predicates in a context, and return results.

```

[77]: class ConcolicTracer(ConcolicTracer):
    def zeval(self, predicates=None):
        if predicates is None:
            path = self.path
        else:
            path = list(self.path)
            for i in sorted(predicates):
                if len(path) > i:
                    path[i] = predicates[i]
                else:
                    path.append(predicates[i])

        r, sol = zeval_py(path, self)
        if r == 'sat':
            return r, {k: sol.get(self.fn_args[k], None) for k in self.fn_args}
        else:
            return r, None

```

Given a set of predicates that the function encountered, and the tracer under which the function was executed, the `zeval_py()` function first declares the relevant symbolic variables, and uses the `z3.Solver()` to provide a set of inputs that would trace the same path through the function.

```

[78]: def zeval_py(path, cc):
    for decl in cc.decls:
        if cc.decls[decl] == 'BitVec':
            v = "z3.%s('%s', 8)" % (cc.decls[decl], decl)
        else:
            v = "z3.%s('%s')" % (cc.decls[decl], decl)
    exec(v)
    s = z3.Solver()
    s.add(z3.And(path))
    if s.check() == z3.unsat:
        return 'No Solutions', {}
    elif s.check() == z3.unknown:
        return 'Gave up', None
    assert s.check() == z3.sat

```

```
m = s.model()
return 'sat', {d.name(): m[d] for d in m.decls()}
```

It can be used as follows:

```
[79]: with ConcolicTracer() as _:
      _[factorial](5)
```

```
[80]: _.zeval()
```

```
[80]: ('sat', {'n': 5})
```

That is, given the set of constraints, the assignment `n == 5` conforms to all constraints.

### 1.3.13 DSE

Let's now see how we can explore an example program using DSE.

```
[81]: def triangle(a, b, c):
      if a == b:
          if b == c:
              return 'equilateral'
          else:
              return 'isosceles1'
      else:
          if b == c:
              return 'isosceles2'
          else:
              if a == c:
                  return 'isosceles3'
              else:
                  return 'scalene'
```

```
[82]: triangle(1, 2, 1)
```

```
[82]: 'isosceles3'
```

DSE would start with arbitrary inputs, which are executed concolically.

```
[83]: with ConcolicTracer() as _:
      print(_[triangle](1, 2, 3))
```

```
scalene
```

The symbolic variables are as follows:

```
[84]: _.decls
```

```
[84]: {'triangle_a_int_1': 'Int',
      'triangle_b_int_2': 'Int',
```

```
'triangle_c_int_3': 'Int'}
```

The predicates are as follows:

```
[85]: _.path
```

```
[85]: [Not(triangle_a_int_1 == triangle_b_int_2),  
       Not(triangle_b_int_2 == triangle_c_int_3),  
       Not(triangle_a_int_1 == triangle_c_int_3)]
```

Using `zeval()`, we solve these path conditions and obtain a solution. We find that Z3 gives us three distinct integer values:

```
[86]: _.zeval()
```

```
[86]: ('sat', {'a': 0, 'b': -2, 'c': -1})
```

If we invoke `triangle()` with these very values, we take the exact same path as the original input:

```
[87]: triangle(0, -2, 1)
```

```
[87]: 'scalene'
```

The next step consists of picking one of the conditions of the path condition, and negating it.

```
[88]: z3.Not(_.path[2])
```

```
[88]: Not(Not(triangle_a_int_1 == triangle_c_int_3))
```

A solution to the negated path condition is a new test input that will follow a different execution path.

```
[89]: _.zeval({2: z3.Not(_.path[2])})
```

```
[89]: ('sat', {'a': 1, 'b': 0, 'c': 1})
```

```
[90]: with ConcolicTracer() as _:  
       print(_[triangle](1, 0, 1))
```

isosceles3

```
[91]: _.path
```

```
[91]: [Not(triangle_a_int_1 == triangle_b_int_2),  
       Not(triangle_b_int_2 == triangle_c_int_3),  
       triangle_a_int_1 == triangle_c_int_3]
```

DSE now continues replacing and negating individual conditions.

```
[92]: _.path.pop()  
       _.zeval({1: z3.Not(_.path[1])})
```

```
[92]: ('sat', {'a': 1, 'b': 0, 'c': 0})
```

```
[93]: with ConcolicTracer() as _:  
      print(_[triangle](1, 0, 0))
```

isosceles2

```
[94]: _.path
```

```
[94]: [Not(triangle_a_int_1 == triangle_b_int_2),  
      triangle_b_int_2 == triangle_c_int_3]
```

```
[95]: _.zeval({0: z3.Not(_.path[0])})
```

```
[95]: ('sat', {'a': 0, 'b': 0, 'c': 0})
```

```
[96]: triangle(0, 0, 0)
```

```
[96]: 'equilateral'
```

To do this systematically we need a (random) starting point:

```
[97]: import random  
  
fn = triangle  
args = []  
for param in inspect.signature(fn).parameters:  
    args.append(random.randrange(1000))  
  
print(args)
```

```
[645, 952, 654]
```

Now we can iteratively explore the paths of the program.

```
[98]: import copy  
  
def dse(fn, args):  
    explored_paths = set()  
    tests = []  
    worklist = [args]  
    while worklist:  
        args = worklist.pop()  
        with ConcolicTracer() as _:  
            ret = _[fn](*args)  
            if str(_.path) in explored_paths:  
                continue  
            explored_paths.add(str(_.path))  
            tests.append((args, ret))
```

```

        for index, cond in enumerate(_path):
            new_path = copy.copy(_path)
            new_path[index] = z3.Not(_path[index])
            if str(new_path) not in explored_paths:
                r, result = zeval_py(new_path, _)
                if r == 'sat':
                    args = [ result.get(_fn_args[k]).as_long() for k in _
↪fn_args]

                    worklist.append(args)

    return tests

```

```
[99]: dse(triangle, args)
```

```

[99]: [([645, 952, 654], 'scalene'),
      ([1, 0, 1], 'isosceles3'),
      ([1, 0, 0], 'isosceles2'),
      ([0, 0, 0], 'equilateral'),
      ([1, 1, 0], 'isosceles1')]

```

You can find more details on how to do this in the [Fuzzing Book](#).